



Electronic Design Workshop Project

Solar Irradiance Meter

Archit Gupta 2023UEC2558

Debasish 2023UEC2571

Ayush Raj 2023UEC2575

April 2025

Index

1. Acknowledgement
2. Materials Required
3. Description to the Sensors Used
4. Schematic Diagram
5. Arduino IDE Code
6. Project Description
7. Result and Conclusion

Acknowledgement

We sincerely thank our professor, Prof. D.V. Gadre Sir for their guidance and support throughout this project.

We are grateful to the lab staff for providing the necessary resources.

This project enhanced our understanding of electronic design and practical implementation.

We appreciate the teamwork and effort contributed by all members.

Finally, we thank the department for this valuable learning opportunity.

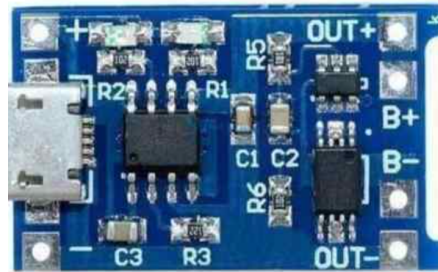
Materials Required

- 3.7V Battery
- TP4056 Charging Module
- Boost Converter Circuit (XL6009E1)
- Arduino Nano
- ACS712 Current Sensor
- BH1750 Light Sensor
- Voltage Divider Circuit
- I2C LCD Display
- Jumper Wires
- Zero PCB Board
- Solar Panel 0.5W
- Push Button
- Solder Wire
- Enclosure for the Project

Description to the Sensors Used

Battery Charging Module - TP4056

The TP4056 is a linear lithium-ion battery charger module. It includes overvoltage, overcurrent, and overtemperature protection. It is used to safely charge 3.7V Li-ion batteries from USB or micro-USB input. It also features auto-cutoff when the battery is fully charged.



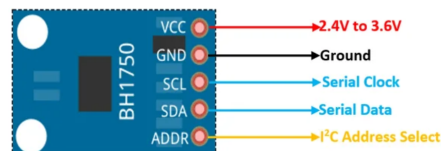
Current Sensor - ACS712

The ACS712 is a Hall effect-based current sensor that can measure both AC and DC current. It provides an analog voltage output that is proportional to the current. It is used to measure the current flow from the solar panel, helping in calculating the power output.



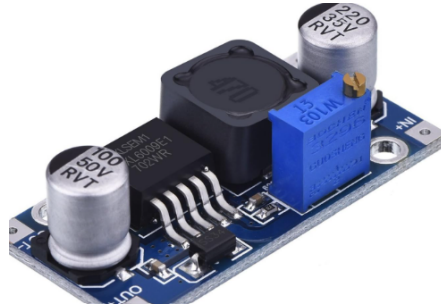
Ambient Light Sensor - BH1750

The BH1750 is a digital lux sensor communicating via I2C. It directly outputs light intensity in lux, making it ideal for solar irradiance applications. It is accurate, compact, and easy to use with Arduino.

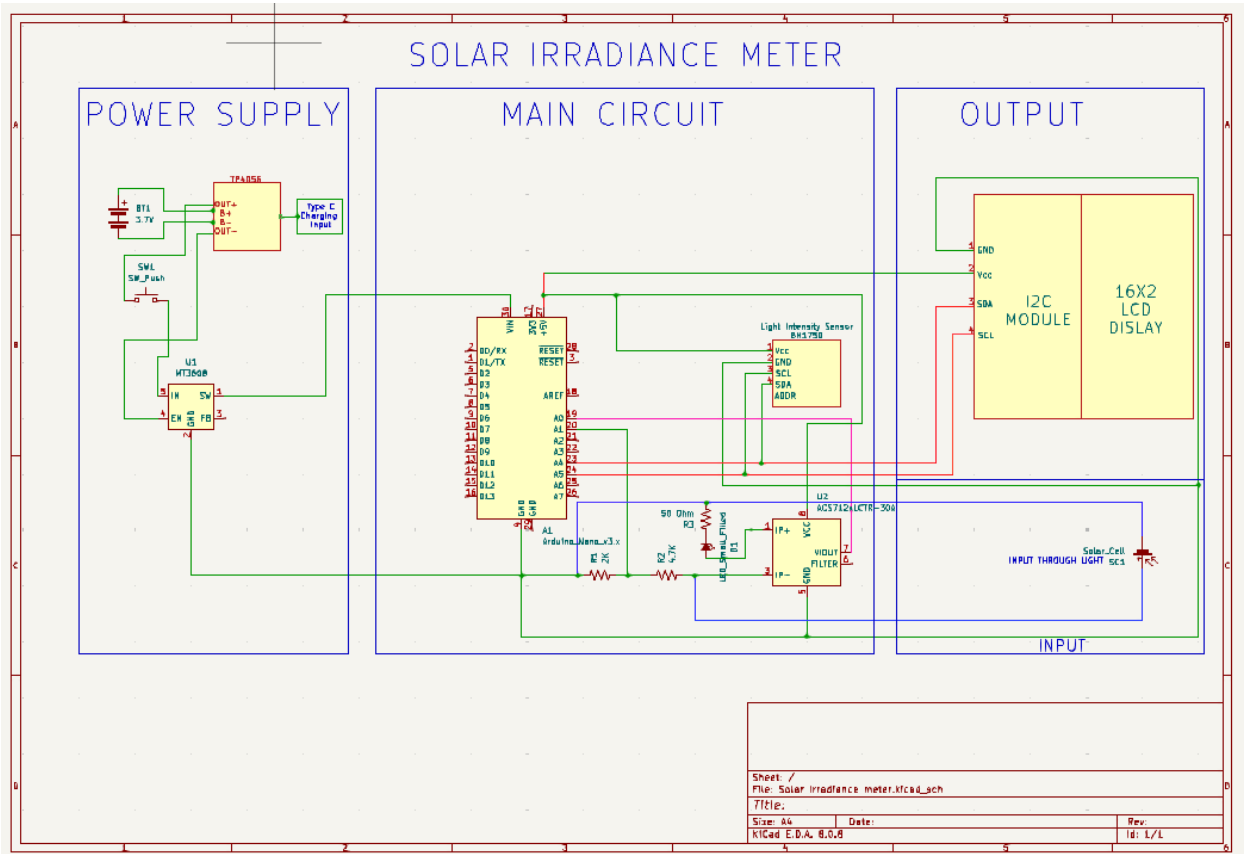


Boost Module - XL6009E1

The XL6009E1 is a DC-DC step-up converter that increases the battery voltage (for example, 3.7V) to 5V. This 5V is necessary for running the Arduino Nano and other modules like LCD and sensors. It ensures a consistent voltage even as the battery is discharged.



Schematic Diagram



Arduino IDE Code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <ACS712.h>
#include <EEPROM.h>
#include <BH1750.h>

#define LCD_ADDRESS 0x27
#define LCD_COLUMNS 16
#define LCD_ROWS 2
#define LCD_REFRESH_MS 1000

#define EEPROM_SIGNATURE_ADDR 0
#define EEPROM_MIDPOINT_ADDR 2
#define EEPROM_SIGNATURE 0xAB42

#define CURRENT_PIN A0
#define VOLTAGE_PIN A1
#define BUTTON_PIN 2

#define ADC_VREF 5.0
#define ADC_STEPS 1023
#define VOLTAGE_DIV_RATIO 3.361

ACS712 acs(CURRENT_PIN, ADC_VREF, ADC_STEPS, 100);
LiquidCrystal_I2C lcd(LCD_ADDRESS, LCD_COLUMNS, LCD_ROWS);
BH1750 lightMeter;

bool manualCalibrationRequested = false;
bool lastButtonState = HIGH;
unsigned long lastLcdUpdate = 0;

void setup() {
  Serial.begin(9600);
  pinMode(BUTTON_PIN, INPUT_PULLUP);

  Wire.begin();
  lcd.init();
  lcd.backlight();
  lcd.clear();

  if (!lightMeter.begin(BH1750::CONTINUOUS_HIGH_RES_MODE)) {
    lcd.print("BH1750 ERROR!");
    while (1);
  }

  loadCalibration();

  if (acs.getMidPoint() == 0) {
    lcd.setCursor(0, 0);
    lcd.print("AUTO CALIB...");
    acs.autoMidPoint();
  }
```



```

        saveCalibration();
        delay(1500);
    } else {
        lcd.setCursor(0, 0);
        lcd.print("MID OFFSET:");
        lcd.setCursor(0, 1);
        lcd.print(acs.getMidPoint());
        delay(1500);
    }

    lcd.clear();
    lcd.setCursor(2, 0);
    lcd.print("SYSTEM READY");
    delay(1000);
}

void loop() {
    checkCalibrationButton();

    if (manualCalibrationRequested) {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("CALIBRATING...");
        Serial.println("Manual calibration triggered");

        acs.setMidPoint(0);
        delay(100);
        acs.autoMidPoint();
        saveCalibration();

        int mid = acs.getMidPoint();
        lcd.setCursor(0, 1);
        lcd.print("MID:");
        lcd.print(mid);
        Serial.print("New midpoint: ");
        Serial.println(mid);

        delay(2000);
        lcd.clear();
        manualCalibrationRequested = false;
    }

    if (millis() - lastLcdUpdate >= LCD_REFRESH_MS) {
        updateDisplay();
        lastLcdUpdate = millis();
    }
}

void updateDisplay() {
    float vout = analogRead(VOLTAGE_PIN) * (ADC_VREF / ADC_STEPS);
    float voltage = vout * VOLTAGE_DIV_RATIO;

    float current = voltage / 50.0;
    float power = voltage * current;

```

```

float lux = lightMeter.readLightLevel();

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("V:");
lcd.print(voltage, 2);
lcd.print(" I:");
lcd.print(current, 3);

lcd.setCursor(0, 1);
lcd.print("P:");
lcd.print(power, 2);
lcd.print("W Lx:");
lcd.print((int)lux);

Serial.print("Voltage: ");
Serial.print(voltage, 2);
Serial.print(" V | Current: ");
Serial.print(current, 3);
Serial.print(" A | Power: ");
Serial.print(power, 2);
Serial.print(" W | Lux: ");
Serial.println((int)lux);
}

void checkCalibrationButton() {
  bool currentButtonState = digitalRead(BUTTON_PIN);
  if (currentButtonState == LOW && lastButtonState == HIGH) {
    delay(50);
    if (digitalRead(BUTTON_PIN) == LOW) {
      manualCalibrationRequested = true;
    }
  }
  lastButtonState = currentButtonState;
}

void saveCalibration() {
  int currentMid = acs.getMidPoint();
  int storedMid;
  EEPROM.get(EEPROM_MIDPOINT_ADDR, storedMid);
  if (storedMid != currentMid) {
    EEPROM.put(EEPROM_SIGNATURE_ADDR, EEPROM_SIGNATURE);
    EEPROM.put(EEPROM_MIDPOINT_ADDR, currentMid);
    Serial.print("Calibration saved to EEPROM. Midpoint: ");
    Serial.println(currentMid);
  }
}

void loadCalibration() {
  int sig;
  EEPROM.get(EEPROM_SIGNATURE_ADDR, sig);
  if (sig == EEPROM_SIGNATURE) {
    int midpoint;
    EEPROM.get(EEPROM_MIDPOINT_ADDR, midpoint);
  }
}

```

```
    acs.setMidPoint(midpoint);  
    Serial.print("Loaded midpoint from EEPROM: ");  
    Serial.println(midpoint);  
  } else {  
    acs.setMidPoint(0);  
    Serial.println("No valid calibration found. Using default midpoint.");  
  }  
}
```

Project Description

1. A 3.7V Li-ion battery is used as the primary power source for the system, enabling portability and off-grid operation.
2. The TP4056 module is integrated for safe charging of the battery through a micro-USB port, with protection features.
3. The XL6009 boost converter steps up the battery voltage to a steady 5V to run all 5V-compatible modules.
4. A solar panel is connected to demonstrate real-time energy harvesting under sunlight.
5. The voltage of the solar panel is measured using a voltage divider, which scales it safely for Arduino input.
6. The voltage divider circuit is effectively used as a voltage sensor, allowing the Arduino to measure the solar panel voltage accurately within its ADC range.
7. The ACS712 current sensor is used to measure the current drawn by the load or coming from the solar panel.
8. The BH1750 sensor measures ambient light in lux, representing solar irradiance accurately.
9. The Arduino Nano reads analog values from the voltage and current sensors and digital lux data over I2C.
10. Real-time power output is calculated using the formula $P = V \times I$.
11. All measured and calculated data are displayed on a 16x2 I2C LCD screen for easy reading.
12. The I2C interface allows using only two wires (SCL and SDA), simplifying the circuit.
13. The LCD backlight improves visibility in outdoor conditions during sunlight testing.
14. Enclosure is added to secure the electronics and make the device robust and field-ready.
15. A push button is being used to turn the system on or off manually, adding control flexibility.
16. The complete setup provides a reliable, low-cost tool to analyze solar energy potential in any environment.

Result and Conclusion

The Solar Irradiance Meter project was successfully designed and implemented. The system is capable of accurately measuring:

- The voltage generated by the solar panel
- The current drawn by the connected load
- The power output derived from solar energy (calculated in real-time)

The readings are displayed on an I2C LCD screen, making it user-friendly and suitable for quick field assessments or laboratory analysis.

We conclude that this Solar Irradiance Meter serves as a reliable tool for basic solar performance testing and educational purposes.